EESC4550/5550: Practical Atmospheric Modeling

Topic 8: Essential Concepts for AI/ML in Atmospheric Science

WONG Hok Fong Silas

2026/27 Term 1



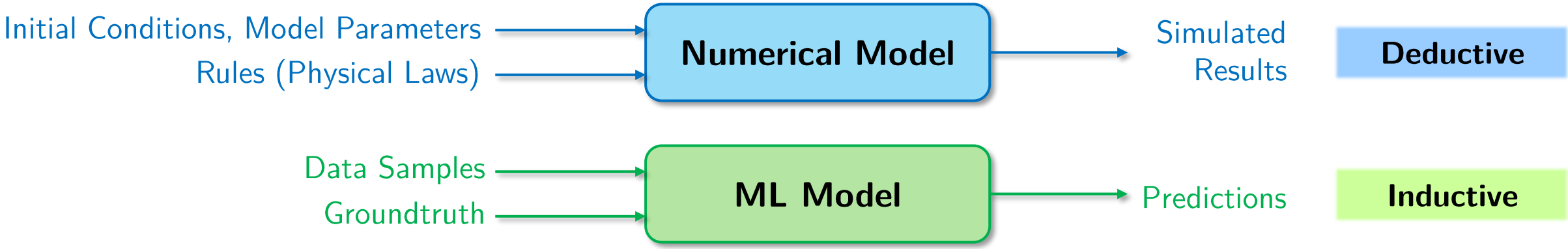


DEPARTMENT OF
EARTH AND ENVIRONMENTAL SCIENCES
地 球 與 環 境 科 學 系

Outline of This Topic

1. Modeling Approaches: Computational versus ML
2. AI, ML, DL, and Data Science
3. Basic Concepts in AI & ML
4. Machine Learning Tools for Atmospheric Science
   a) Regression and Regularization
   b) Tree, Forests, and Boosting
   c) Neural Networks
   d) SHAP Values

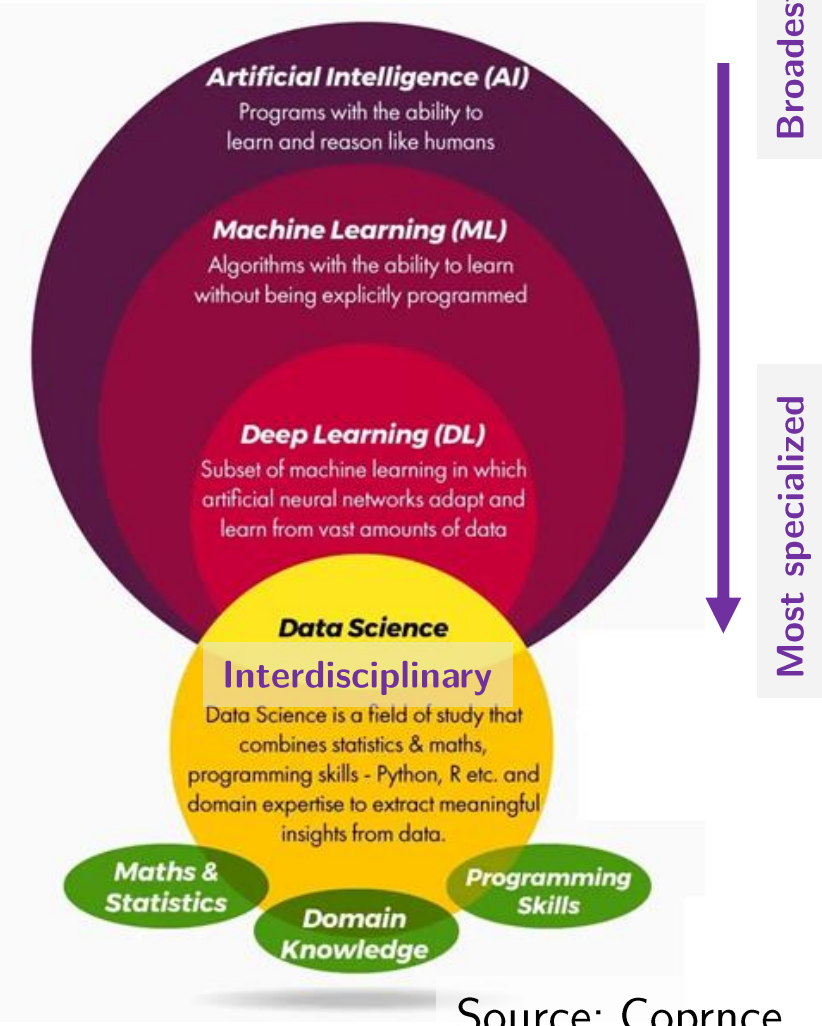
Modeling Approaches: Computational versus ML



- Predictions of atmospheric processes are difficult: The Earth is huge, resolution is limited and we *cannot* represent all important processes within model simulations.
- The Earth System shows "chaotic" dynamics which makes it difficult to predict the future based on equations. All Earth System components (atmosphere, ocean, land surface, cloud physics, ...) are connected in a non-trivial way.
- Some of the processes involved are not well understood. However, we have a huge number of observations and Earth System data.

Relationship between AI, ML, DL, and Data Science

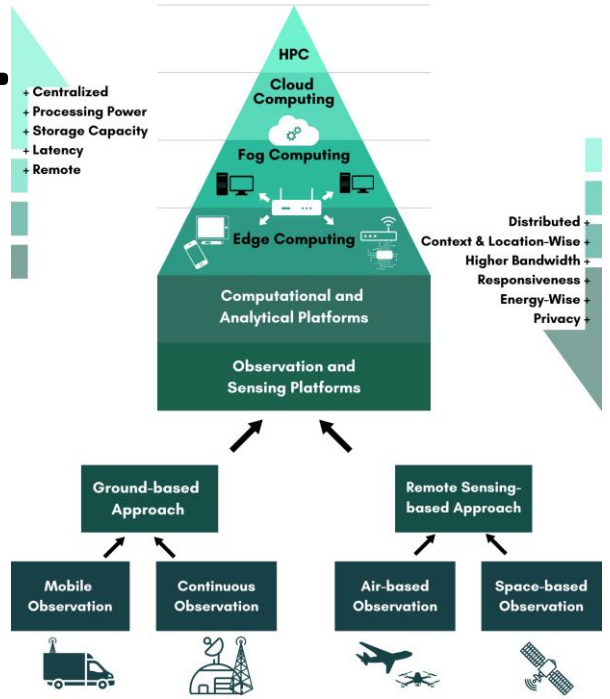
- **Artificial Intelligence**: Computer systems capable of performing complex tasks that mimic human intelligence, such as reasoning, making decisions, or solving problems.
- **Machine Learning**: Algorithms whose performance improve as they are exposed to more data over time.
- **Deep Learning**: A specific group of machine learning models in form of multi-layered neural networks, that can be learnt from massive data.
- **Data Science** integrates all the above to extract insights from data (exploratory data analytics) and make predictions from large datasets (predictive analytics).



Source: Coprnce

The Success of AI & ML

- Increase in data volume
- New computing hardware and availability of compute resources
- Ease of use of modern ML frameworks
- Increase in knowledge



Source: Zaidan et al., 2025

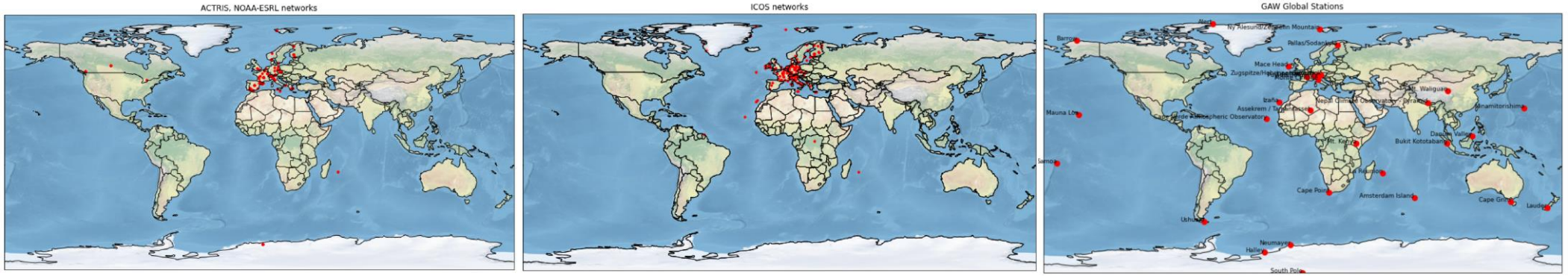
Modern ML Frameworks

scikit learn

PyTorch

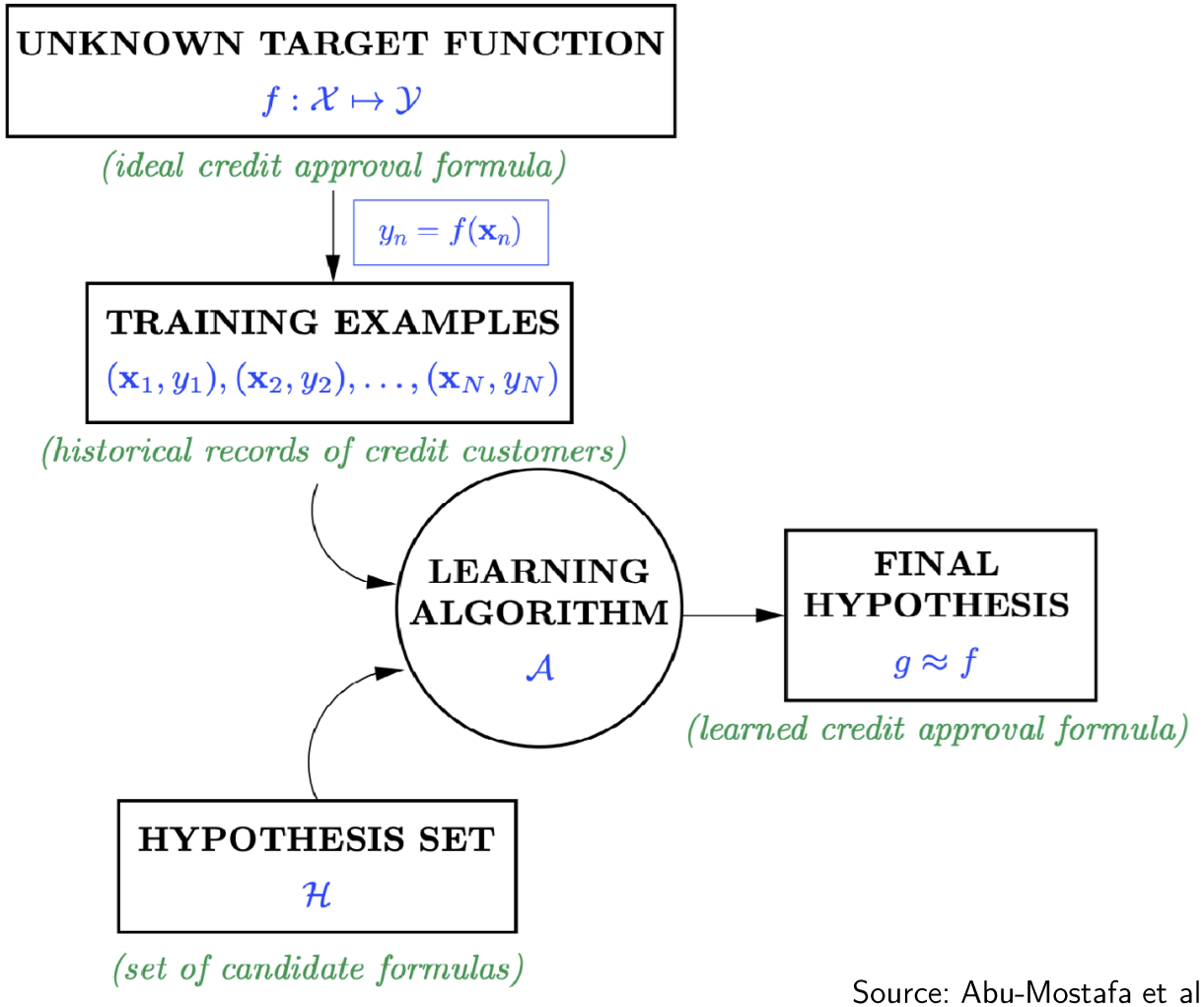
Hugging Face

APACHE Spark



Source: Zaidan et al., 2025

Machine Learning: Learning from Data



Source: Abu-Mostafa et al., *Learning From Data: A Short Course*, 2012

The Three Main Types of Machine Learning

Supervised Learning

Learns a function $f : X \to Y$ that maps an input feature vector X to a target output Y, given a labeled training dataset $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$.

Goal: Minimize prediction error (loss) between the predicted output $\hat{y} = f(x)$ and the true label y.

Unsupervised Learning

Infers a hidden structure or distribution $p(X)$ from an unlabeled training dataset $\mathcal{D} = \{x_1, x_2, \ldots, x_n\}$ without any corresponding target values Y.

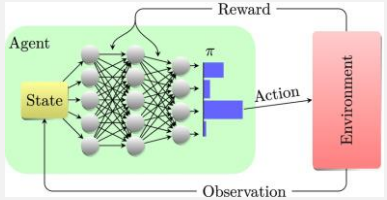
Goal: Maximize data description efficiency, grouping similarity, or density estimation.

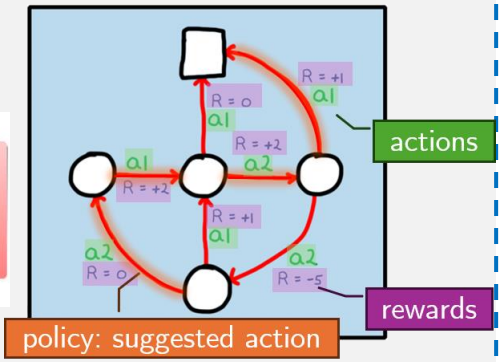
Reinforcement Learning

Modeled as a Markov Decision Process (MDP) where an agent interacts with an environment over discrete time steps, defined by states S, actions A, transition probabilities P, and a scalar reward function R.

Goal: Learn the best series of actions over time by maximizing the expected cumulative discounted reward

$$\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_t\right].$$





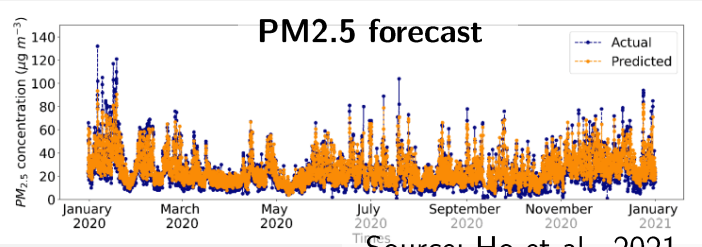
Other Learning Paradigms:

- Semi-Supervised Learning
- Active Learning
- Transfer Learning
- Zero-Shot Learning

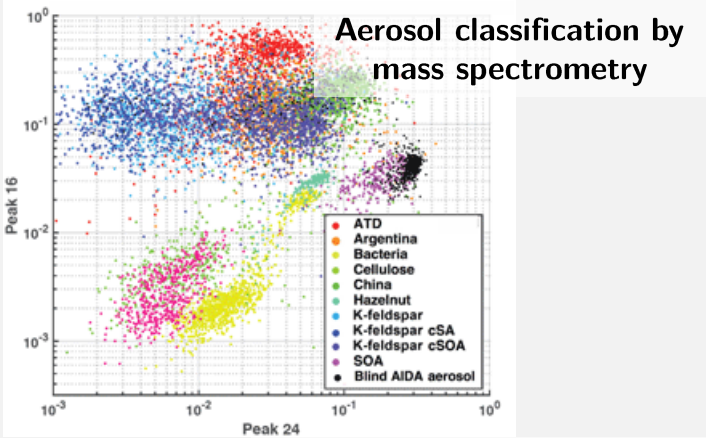
The Three Main Types of Machine Learning

Supervised Learning

Examples:
Classification, Regression



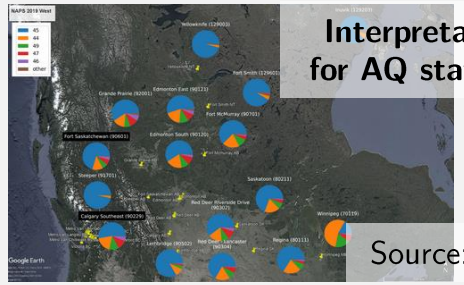
Source: Ho et al., 2021



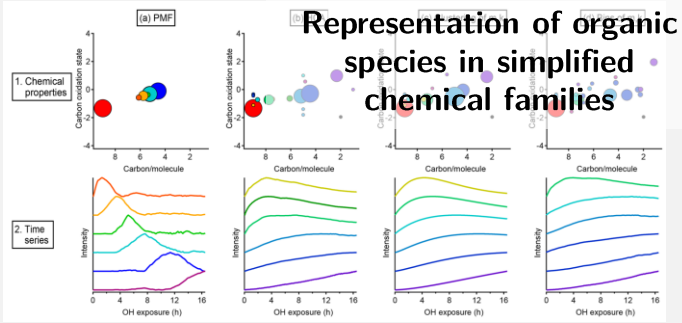
Source: Christopoulos et al., 2018

Unsupervised Learning

Examples:
Clustering, Dimensionality Reduction, Anomaly Detection



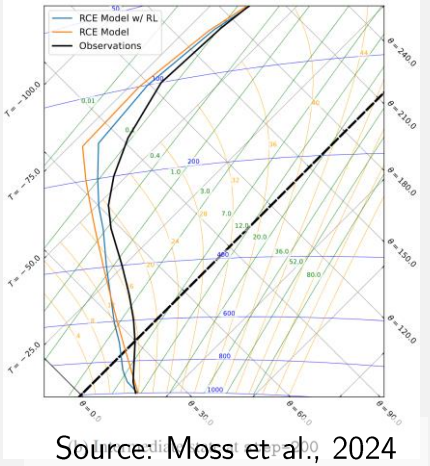
Source: Lee et al., 2026



Source: Koss et al., 2020

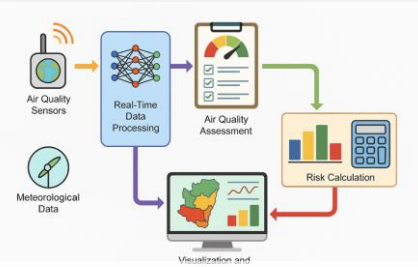
Reinforcement Learning

Examples:
Real-Time Interaction with an Environment (e.g. Game Playing, Robotics, Self-Driving Cars)



Source: Moss et al., 2024

RL for improving sub-grid process parameterizations under physical constraints

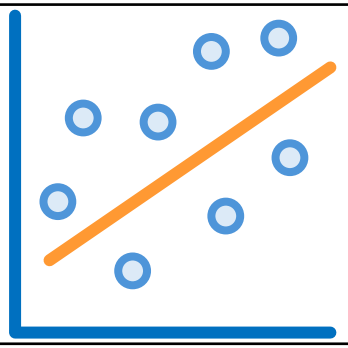
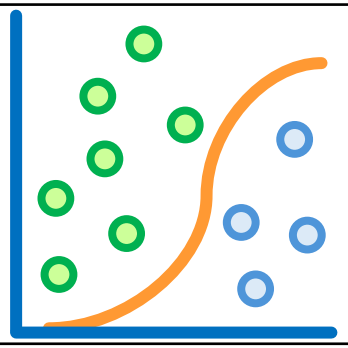


Source: Rajesh et al., 2025

Autonomous policy formulation and execution for air quality control

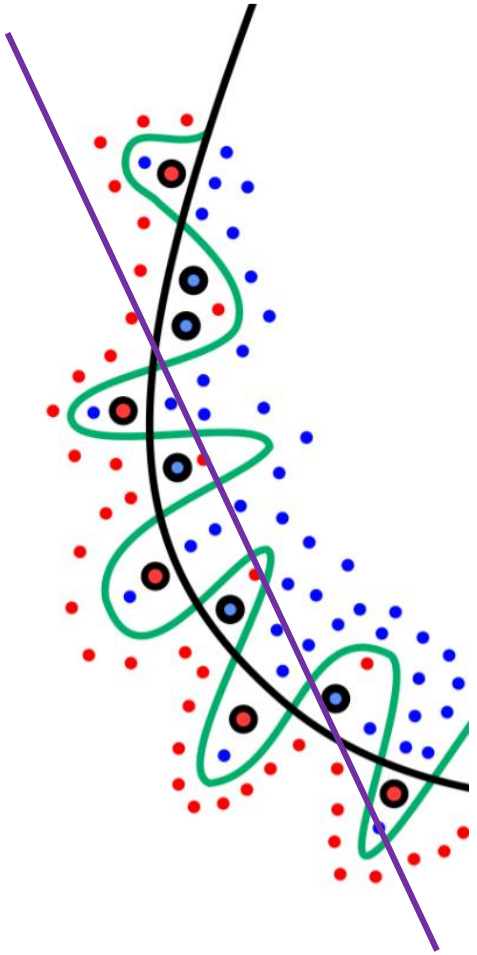
(Sinha et al., 2025)

Classification versus Regression

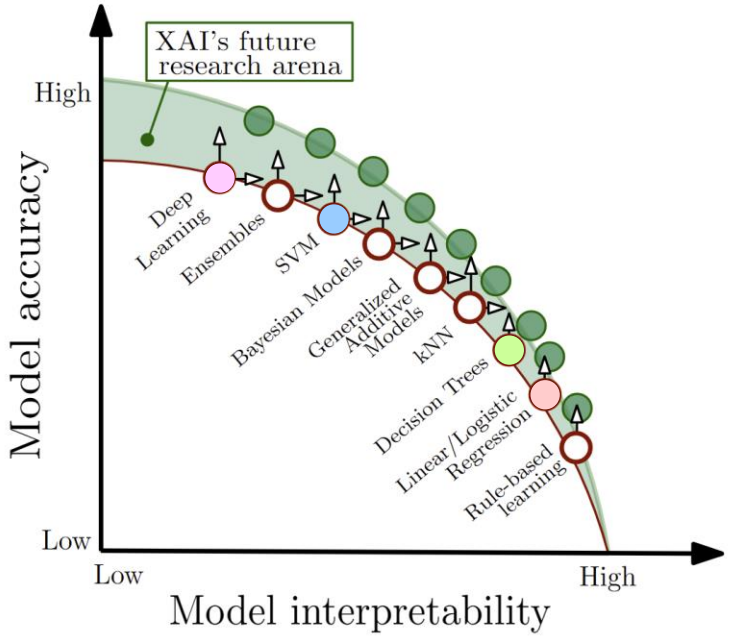
	Classification	Regression
Predict	Label / Class / Category	Numeric value
Method	May consider more abstract and sophisticated models	Mainly least-squares models
Goal	Finding decision boundaries that separate classes	Finding the underlying trend for predicting numerical outcomes
Applications in Atmospheric Science	Aerosol type categorization; Reaction regime mapping; Dust source region mapping	Estimating surface $PM_{2.5}$; Vertical profile reconstruction; Biogenic VOC modeling
Schematics		

Generalization and Overfitting

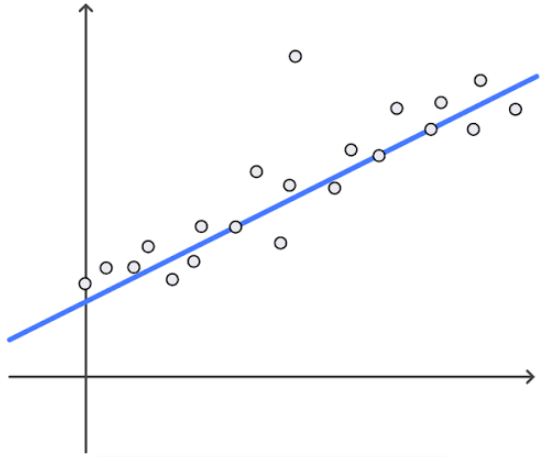
- **Generalization**: how good is a model's performance on unseen data.
- **Underfitting**: Model can't fit the complex data.
- **Overfitting**: Model exactly fits the training data; Does not generalize to unseen data. Can be avoided by *reducing model complexity, regularization, pruning*, etc.



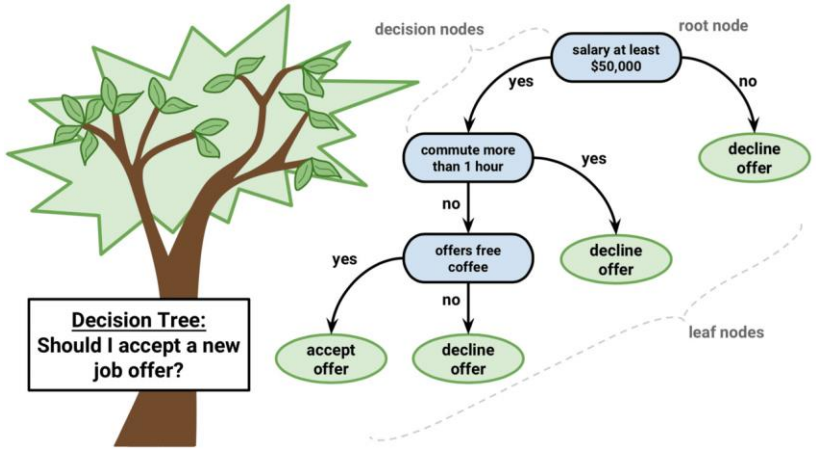
Toolkits for Data Analytics



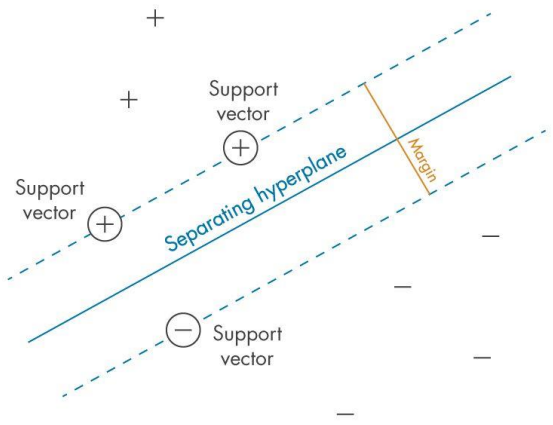
Source: Arrieta et al., 2020



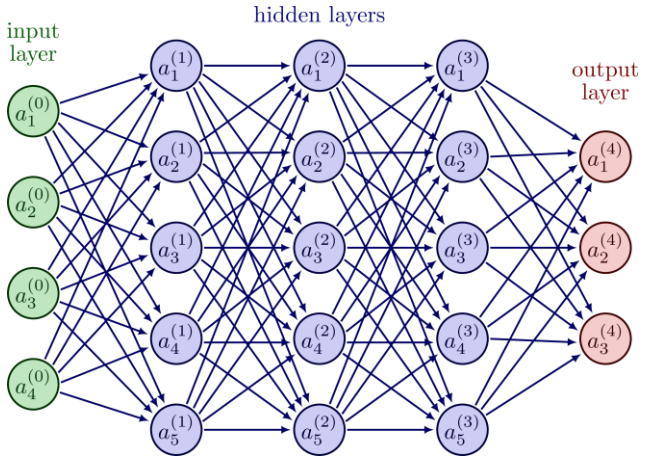
Linear Models



Trees and Forests



Support Vector Machines



Neural Networks

Tabulated Data Representations

- A sample is a vector of predictors (*features*): $\mathbf{x} = (x_1, \ldots, x_p)^\top$. Stack n samples row-wise into the design matrix X.
- Meteorology, emissions and time-of-day all become columns of X; the pollutant concentration is the target vector $\mathbf{y}$.
- Applying weightings $\beta_1, \ldots, \beta_p$ to feature 1 to p, then prediction of sample i is given by

$$\hat{y}_i = \mathbf{x}_i^\top \boldsymbol{\beta} = \beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p$$

$$\hat{\mathbf{y}} = X\boldsymbol{\beta}$$

Bias term

	Features			Target
Sample	Temp (°C)	NOx Emission (ton/day)	Hour (0-24)	Surface PM2.5 conc. (μg/m³)
1	15.2	120	8	12.5
2	18.5	135	12	18.0
3	22.1	110	16	25.3
4	20.0	125	20	21.1

Example

$$\mathbf{x}_1 = [15.2 \quad 120 \quad 8]^\top$$

$$\mathbf{y} = [12.5 \quad 18.0 \quad 25.3 \quad 21.1]^\top$$

$$X = \begin{bmatrix} 15.2 & 120 & 8 \\ 18.5 & 135 & 12 \\ 22.1 & 110 & 16 \\ 20.0 & 125 & 20 \end{bmatrix}$$

Think: How to include a *bias term* by alternating the design matrix?

Linear Regression

- **Problem**: Given a set of 2D data points

$$\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}.$$

  x is the explanatory variable, and y is the dependent variable.

- **Goal**: Predict the y values using the x values with minimized sum of squared errors.

$$RSS = \sum_{i=1}^{n} (y_i - a - bx_i)^2$$

  To achieve this, we consider fitting a line through the data

$$\boxed{y = a + bx}$$

  where a and b are some parameters to be determined.

Linear Regression

Note that we can rewrite the problem as a matrix problem:

$$\begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}.$$

Recall that finding the least-squares solution to $A\mathbf{x} = \mathbf{b}$ is equivalent to solving the normal equation $A^\top A\mathbf{x} = A^\top \mathbf{b}$. Therefore, we have

$$\boxed{\begin{bmatrix} a \\ b \end{bmatrix} = (M^\top M)^{-1} M^\top \mathbf{y}}$$

where

$$M = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix} \quad \text{and} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}.$$

Multiple Linear Regression (MLR)

- **Problem**: Suppose that we have d explanatory variables and one dependent variables instead, $\mathcal{D} = (\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_n, y_n)$.
- **Goal**: We wish to fit the data with the function

$$y = a_0 + a_1 x_1 + a_2 x_2 + \ldots + a_d x_d.$$

with the residual sum of squares

$$RSS = \sum_{i=1}^{n} (y_i - a_0 - a_1 x_{i,1} - a_2 x_{i,2} - \ldots - a_d x_{i,d})^2$$

Multiple Linear Regression (MLR)

Formulate the problem with the matrix notation:

$$\begin{bmatrix} 1 & x_{1,1} & x_{1,2} & \cdots & x_{1,d} \\ 1 & x_{2,1} & x_{2,2} & \cdots & x_{2,d} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n,1} & x_{n,2} & \cdots & x_{n,d} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_d \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}.$$

From the normal equations, the best-fit parameters are given by:

$$\boxed{\begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_d \end{bmatrix} = (M^T M)^{-1} M^T \mathbf{y},}$$

$$\text{where} \quad M = \begin{bmatrix} 1 & x_{1,1} & x_{1,2} & \cdots & x_{1,d} \\ 1 & x_{2,1} & x_{2,2} & \cdots & x_{2,d} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n,1} & x_{n,2} & \cdots & x_{n,d} \end{bmatrix} \quad \text{and} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}.$$

Polynomial Regression

- **Homework**

Given a set of 2D data points $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$
Fit the data with a degree-k polynomial

$$y = f(x) = c_0 + c_1 x + c_2 x^2 + \ldots + c_k x^k$$

that minimizes

$$RSS = \sum_{i=1}^{n} (y_i - c_0 - c_1 x_i - c_2 x_i^2 - \ldots - c_k x^k)$$

Find the analytical solution that solves for the best-fit parameters.

Weighted Least-Squares

- **Homework**

Given a set of $(d+1)$-D data points $\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_n, y_n)\}$.
Fit the data with the function

$$y = a_0 + a_1 x_1 + a_2 x_2 + \ldots + a_d x_d.$$

that minimizes

$$RSS = \sum_{i=1}^{n} w_i^2 \left(y_i - \hat{y}_i\right)^2 = \sum_{i=1}^{n} w_i^2 \left(y_i - c_0 - c_1 x_{i,1} - \ldots - c_d x_{i,d}\right)^2.$$

Find the analytical solution that solves for the best-fit parameters.

Model Performance Statistics: R^2, RMSE, MAE

Coefficient of determination R^2

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y}_i)^2} = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

$R^2 = 1$ perfect fit; model explains all variance

$R^2 = 0$ no better than predicting the mean

Error metrics

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}, \quad \text{MAE} = \frac{1}{n}\sum_i |y_i - \hat{y}_i|$$

RMSE penalises large errors (squared) – sensitive to outliers/peaks

MAE average absolute error – robust, easy to interpret

Regularization

- Adding constraints or penalties to a model to stop it from overfitting and to improve generalization.

$$\hat{\theta} = \arg\min_{\boldsymbol{\theta}} \left[\frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(y_i, f_{\boldsymbol{\theta}}(\mathbf{x}_i)) + \lambda R(\boldsymbol{\theta}) \right]$$

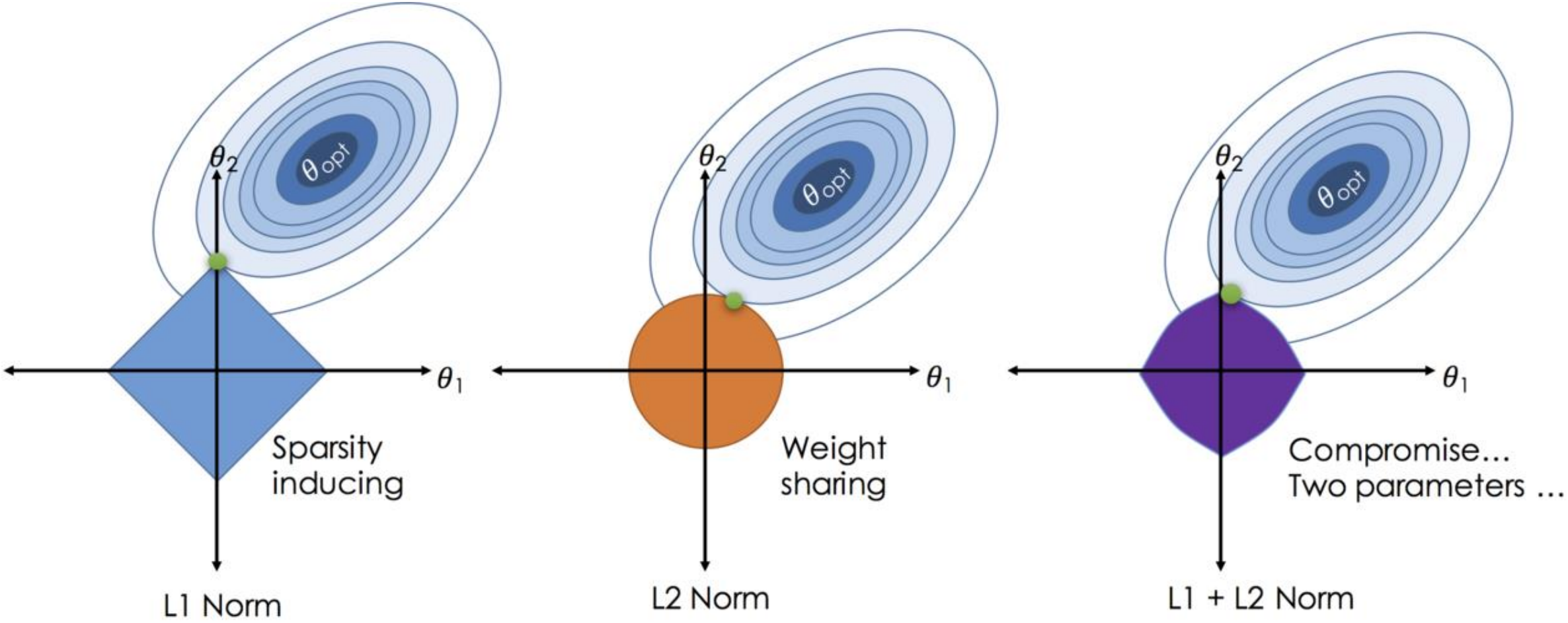
Regularization penalty

Loss funct.

Regularization strength

- **Ridge** shrinks coefficients toward zero. With correlated features, it tends to split weight among them rather than picking one. It reduces variance but keeps all features.
- **Lasso** forces some coefficients to exactly zero. With correlated features, it often selects one arbitrarily and ignores the rest. That selection can change completely with a new sample.
- **Elastic Net** combines both: it selects groups of correlated features together, which is usually more stable and interpretable.

Effect of Lasso, Ridge, and Elastic Net



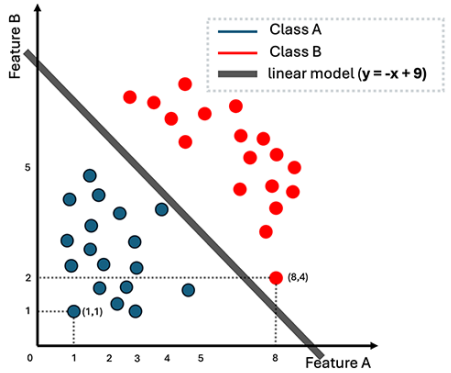
Lasso: $\|y - X\beta\|_2^2 + \lambda\|\beta\|_1$ Ridge: $\|y - X\beta\|_2^2 + \lambda\|\beta\|_2^2$ Elastic Net: $\|y - X\beta\|_2^2 + \lambda\left[\alpha\|\beta\|_1 + (1-\alpha)\|\beta\|_2^2\right]$

Logistic Regression (Optional)

- **Problem**: Suppose that we have d explanatory variables and one dependent variables, $\mathcal{D} = (\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_n, y_n)$, where $y_i \in \{0, 1\}$. Consider the linear classifier

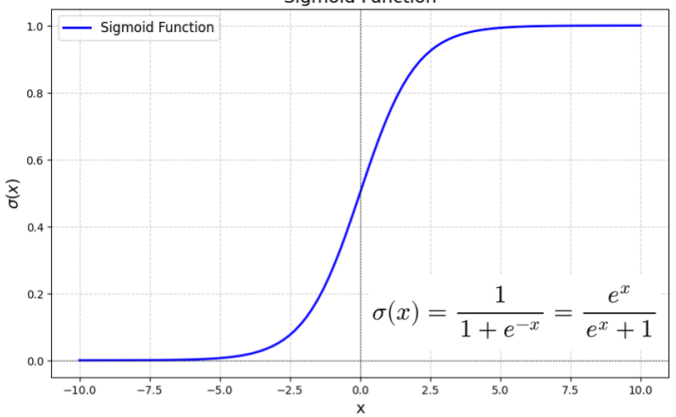
$$z = \mathbf{w}^\top \mathbf{x} + b,$$

$$f_{\mathbf{w},b}(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}.$$



- We aim to minimize the *log loss* function

$$L(\mathbf{w}, b) = \sum_{i=1}^{n} (-y_i \log(f_{\mathbf{w},b}(\mathbf{x}_i)) - (1 - y_i) \log(1 - f_{\mathbf{w},b}(\mathbf{x}_i))).$$



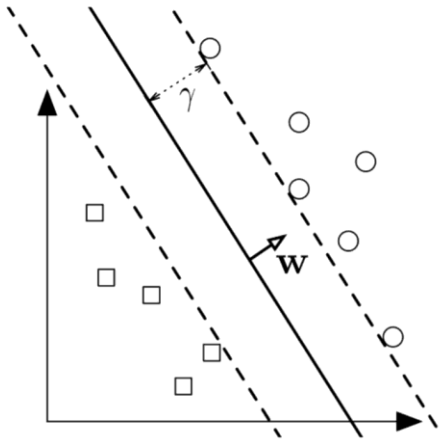
Usually, there is no closed-form solution for the global minimizer
→ Iteratively and numerically solved by gradient-based methods

Think: Why don't we use some seemingly simpler loss functions?

Support Vector Machine (SVM) (Optional)

- Many possible classifiers exists, but they might be too close to the closest points of each class; SVM focuses on the one that maximizes the distance to the closest data points from both classes, i.e., the hyperplane with maximum margin.
- **Problem**: Suppose that we have a $(d+1)$-D *linearly separable dataset* $\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_n, y_n)\}$, with label $y_i \in \{-1, +1\}$ for $i=1 \ldots n$. We find a vector $\mathbf{w}$ and a scalar b that solves the following minimization problem:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{subject to } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \text{ for all } i = 1, 2, \ldots, n$$



Think: What if the data is not linearly-separable?

Soft-Margin SVM (Optional)

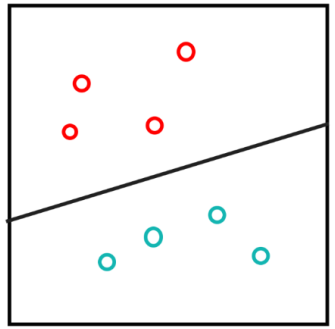
- **Allow misclassification** by adding a function that penalizes the extent of margin violation.

$$\min_{\mathbf{w},b,\zeta_1,\ldots,\zeta_n} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\zeta_i$$

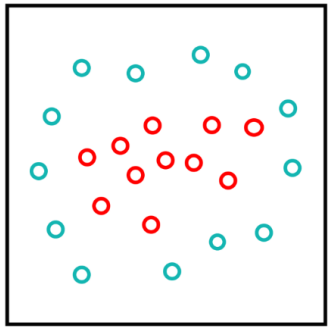
subject to

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \zeta_i,$$
$$\zeta_i \geq 0$$

for all $i = 1, 2, \ldots, n$.



Linearly separable



No good linear separator?

- Note: C performs as a regularization parameter for controlling the trade-off between margin maximization and misclassification penalty.
- ζ_i are slack variables representing the degree of violation of margin by each data point.
- We can also use the L2 formulation $\sum_{i=1}^{n}\zeta_i^2$.

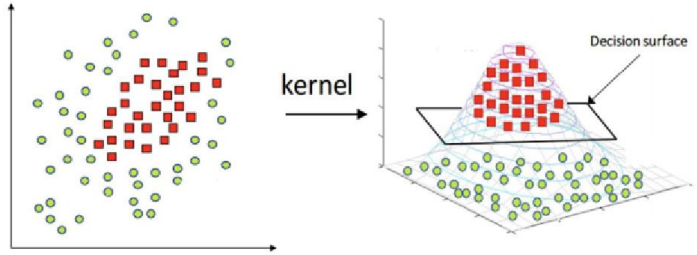
Kernel Transformation and Kernel SVM (Optional)

- Data that is mixed together in lower-dimensional space can be separated by a flat hyperplane when stretched or projected into higher dimensions.

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i(\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}_i) + b) \geq 1$$

Classifier:

$$f(\mathbf{x}) = \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}) + b$$



To apply the kernel trick, we check how similar the new data point is to the critical boundary points.

- Popular choices of *similarity measures*:
  - Polynomial kernel $K(\mathbf{u},\mathbf{v}) = (1 + \mathbf{u}^\top \mathbf{v})^p$
  - Radial Basis Function (RBF/Gaussian) kernel $K(\mathbf{u},\mathbf{v}) = \exp\left(-\|\mathbf{u}-\mathbf{v}\|^2\right)$
  - Sigmoid kernel $K(\mathbf{u},\mathbf{v}) = \tanh\left(p_1 \mathbf{u}^\top \mathbf{v} + p_2\right)$

Note: we maximize the margin between the most similar pair of points on the transformed space.

Interpolation: Ordinary Kriging

- A statistical interpolation method using weighted averages over all control points, with the weights determined by the spatial structure and variability of the data.

$$\min_{\lambda} \mathrm{Var}(Z(\mathbf{x}_0) - \hat{Z}(\mathbf{x}_0)) \qquad \hat{Z}_{OK}(\mathbf{x}_0) = \sum_{i=1}^{n} \lambda_i Z(\mathbf{x}_i), \quad \sum_{i=1}^{n} \lambda_i = 1$$

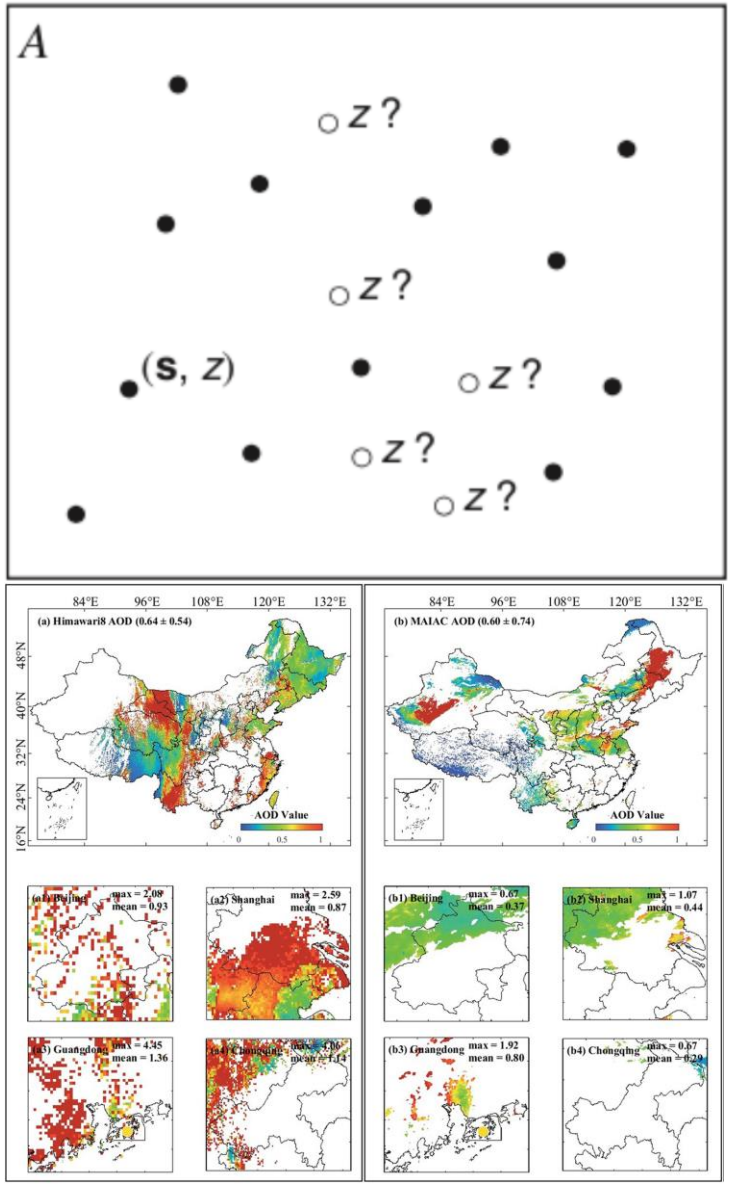
- A **spatial semi-variogram** specifies degree of dissimilarity based on the spatial distance of two data points.

$$\begin{cases} \sum_{j=1}^{n} \lambda_j \gamma(\mathbf{x}_i, \mathbf{x}_j) + \mu = \gamma(\mathbf{x}_i, \mathbf{x}_0), & i = 1, \ldots, n \\ \sum_{j=1}^{n} \lambda_j = 1 \end{cases}$$

A linear system

- **Kriging is a best linear unbiased estimator and also returns an uncertainty at every filled pixel.**

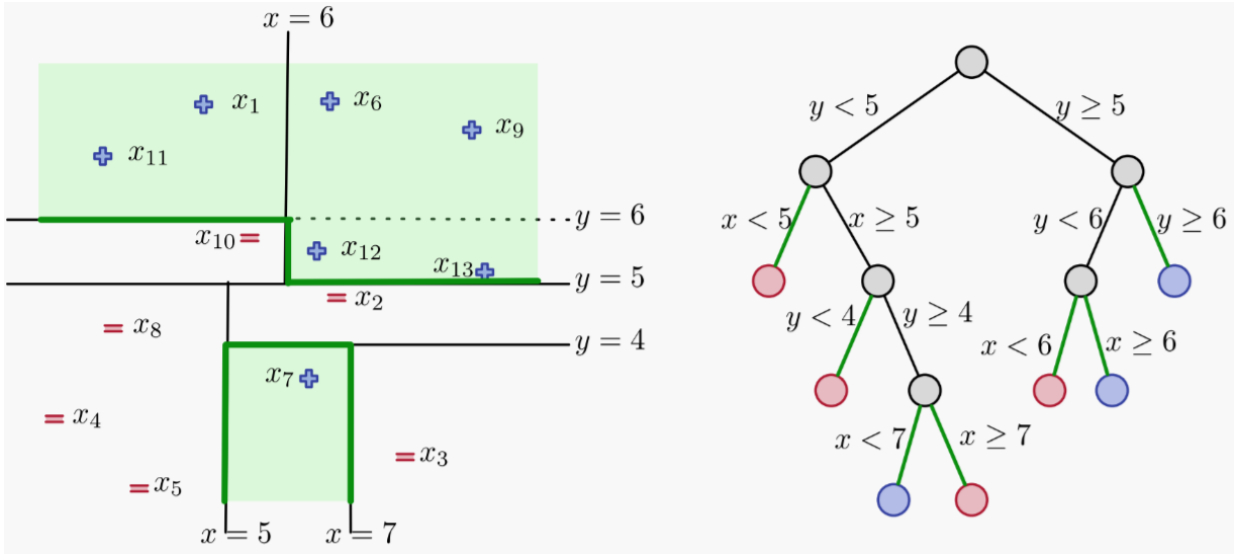
Application: Satellite AOD has 40–80% missing pixels (cloud, bright surfaces). Kriging fills a gap using nearby observed values.

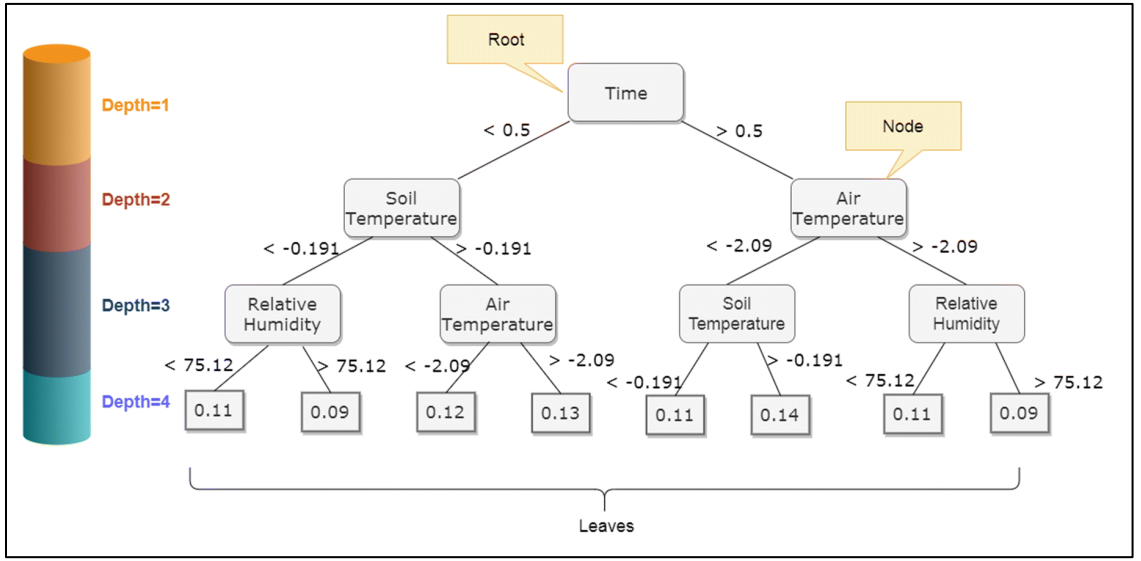


Source: Wang et al., *Journal of Cleaner Production*, 2025

Decision Tree

- A decision tree is a classifier/predictor with binary tree structure for data points $(X,\ y)$.
- Non-overlapping set of subtrees contains disjoint sets of X.





Alumni of CUHK! **Ho Tin Kam**, Random Decision Forests, *Proceedings of 3rd international conference on document analysis and recognition*, 1995

Decision Tree: The Algorithm

- Start with all data in the root node.
- Recurse on the leaf nodes to find the split that most reduces the SSE:
  - Split the node into two children.
  - Recurse on each child.

SSE reduction = variance explained by the split.

- Stop when the depth requirement is met or when there are only one sample left on a node. Each leaf predicts the **mean** of the training responses y in that leaf.

- For a node t with samples S_t, the node impurity is $\mathrm{SSE}(t) = \sum_{i \in S_t} (y_i - \bar{y}_t)^2$
- A split into left S_L and right S_R gives reduction $\Delta\mathrm{SSE} = \mathrm{SSE}(S_t) - [\mathrm{SSE}(S_L) + \mathrm{SSE}(S_R)]$
- To find a split for numeric features, we sort unique values and try the midpoints between consecutive values
- To find a split for categorial features, we sort categories by their mean y and try binary splits between consecutive categories.

Bagging and Boosting

- **Bagging (Bootstrap Aggregating)**
  - **Idea:** Train many trees on **different bootstrap samples** of the data, then average their predictions.

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B} T_b(\mathbf{x})\right) = \frac{1}{B^2}\sum_{b=1}^{B}\mathrm{Var}(T_b) + \frac{1}{B^2}\sum_{b\neq b'}\mathrm{Cov}(T_b, T_{b'})$$

- **Boosting**
  - **Idea:** Train trees **sequentially**. Each new tree focuses on the errors the previous ensemble made. The final prediction is a weighted sum of all trees.
  - For classification: AdaBoost;
  - For differentiable loss functions: Gradient Boosting

$$F_M(\mathbf{x}) = \sum_{m=1}^{M} w_i \cdot h_m(\mathbf{x})$$

XGBoost: eXtreme Gradient Boosting

- Minimize the regularized objective at each round of boosting:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) \qquad \Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

Leaf penalty

Weight penalty

- Approximate by first- and second-order gradient:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^{n} \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i) \right] + \Omega(f_t)$$

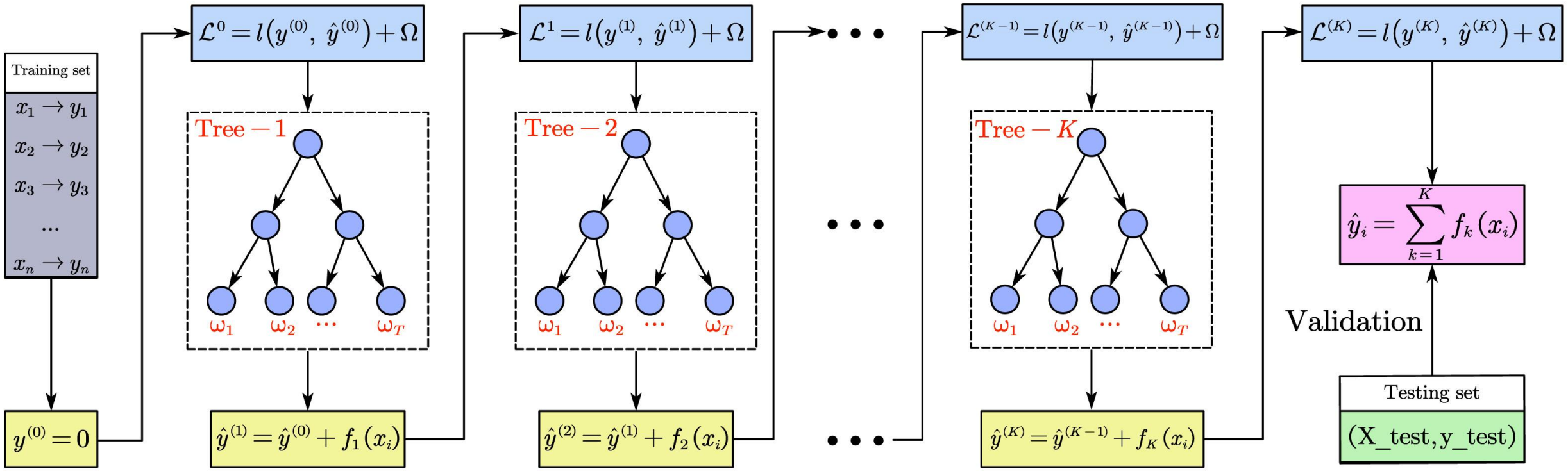
$$g_i = \partial_{\hat{y}} l(y_i, \hat{y}^{(t-1)}) \quad h_i = \partial_{\hat{y}}^2 l(y_i, \hat{y}^{(t-1)})$$

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$$

I_j is the set of samples in leaf j

w_j is the prediction update at leaf j after the j-th weak-learner is constructed

XGBoost: eXtreme Gradient Boosting



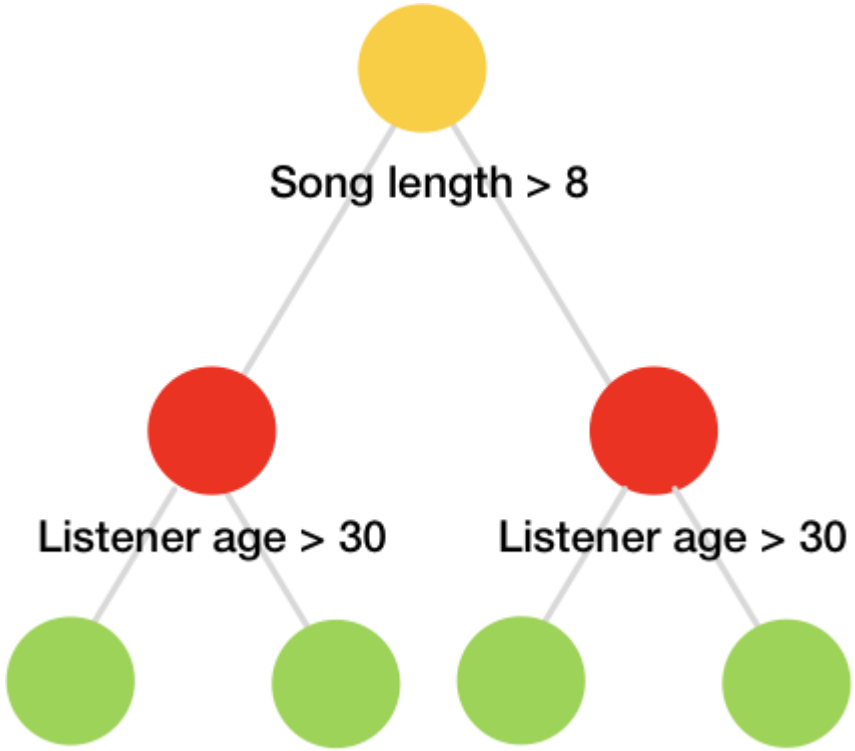
CatBoost

- Standard gradient boosting is trained on residuals computed using the **same data** that the model was fit on, leading to **target leakage** and **prediction shift**.
- CatBoost solves this with **ordered boosting**:
  - Generate a random permutation of the training data.
  - For each sample i, compute the residual using only samples that appear **before** i in the permutation.
  - This ensures the model never uses a sample's own target to predict itself.
- This is particularly important for **small datasets** and **noisy data**, where overfitting is a major risk.

CatBoost

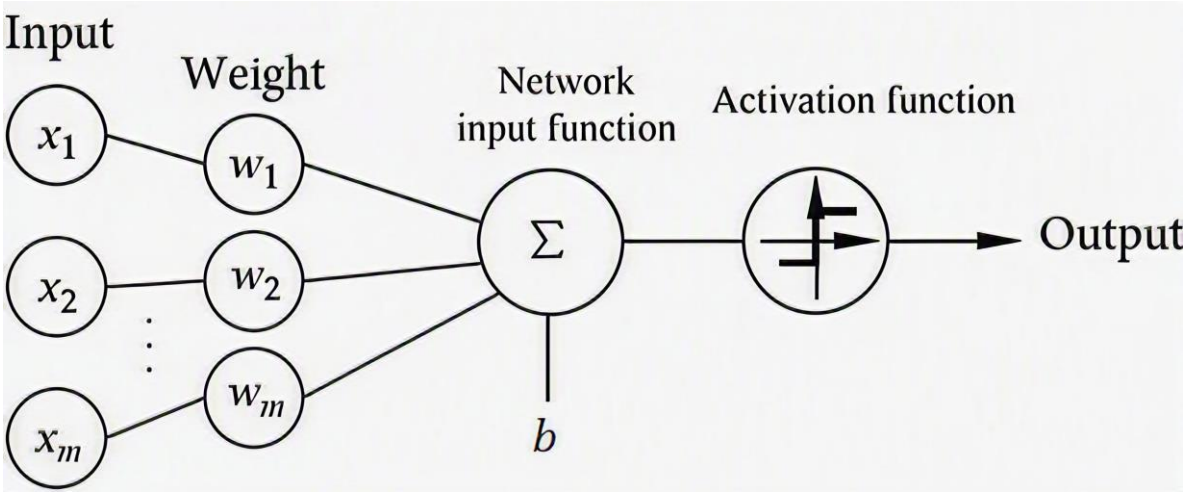
- CatBoost uses **symmetric decision trees**. At every depth level, all decision nodes use the same split condition. No tree traversal is needed.
  - Advantages: faster prediction, regularization, simplicity
  - Disadvantages: expressiveness
- **Categorical feature encoding**:

$$\hat{\mathbf{x}}_k^{(i)} = \frac{\sum_{j<i} \left[\mathbf{x}_k^{(j)} = \mathbf{x}_k^{(i)}\right] \cdot y_j + \omega \cdot p}{\sum_{j<i} \left[\mathbf{x}_k^{(j)} = \mathbf{x}_k^{(i)}\right] + \omega}$$



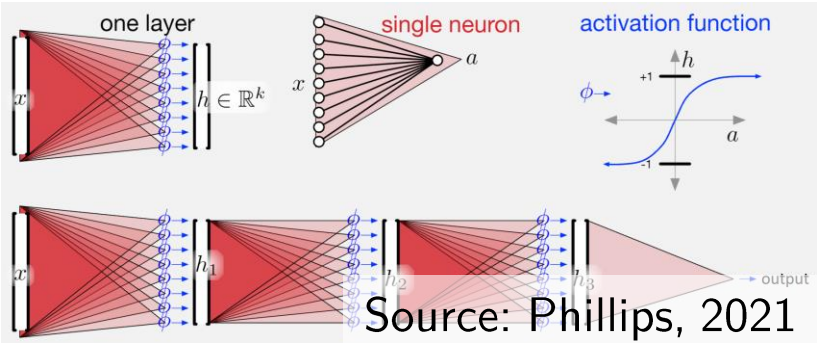
Building Blocks of an NN: Neurons

- A *neuron* is composed of several key components:
  - Input features
    $$\mathbf{x} = (x_1, x_2, \dots, x_n)$$
  - Weights and bias
    $$\mathbf{w} = (w_1, w_2, \dots, w_n) \qquad b$$
  - An activation function, e.g.
    $$h(x) := \mathrm{ReLU}(x) = \max(0, x)$$
    $$h(x) := \sigma(x) = (1 + e^{-x})^{-1}$$
    $$h(x) := \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$
  - An output
    $$f_{\mathbf{w},b}(\mathbf{x}) = h(\mathbf{w}^\top \mathbf{x} + b)$$



Fully Connected Deep Neural Networks

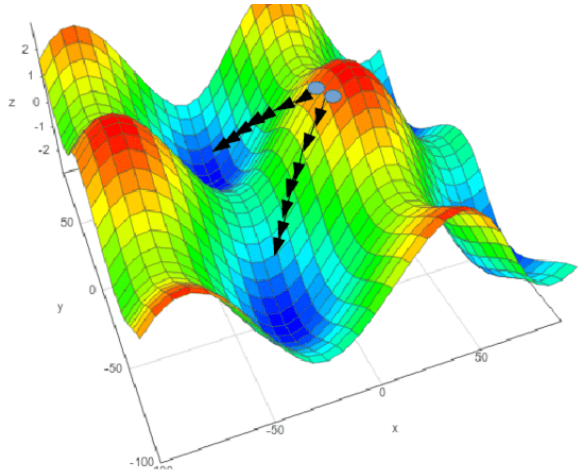
- A deep neural network is composed of a series of *layers*.
- Each layer maps from $\mathbf{h}_\ell \in \mathbb{R}^{k_\ell}$ to $\mathbf{h}_{\ell+1} \in \mathbb{R}^{k_{\ell+1}}$, and is composed of $k_{\ell+1}$ neurons, each followed by an activation function. The input layer is $\mathbf{h}_0 = \mathbf{x}$, the other layers are hidden as the user only provides the input and observes the output.
- For each layer ℓ from 1 to L, the pre-activation vector $z^{(\ell)}$ and post-activation output vector $h^{(\ell)}$ are defined recursively:
  Pre-activation: $\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}$
  Post-activation: $\mathbf{h}^{(\ell)} = \boldsymbol{\phi}^{(\ell)}(\mathbf{z}^{(\ell)})$



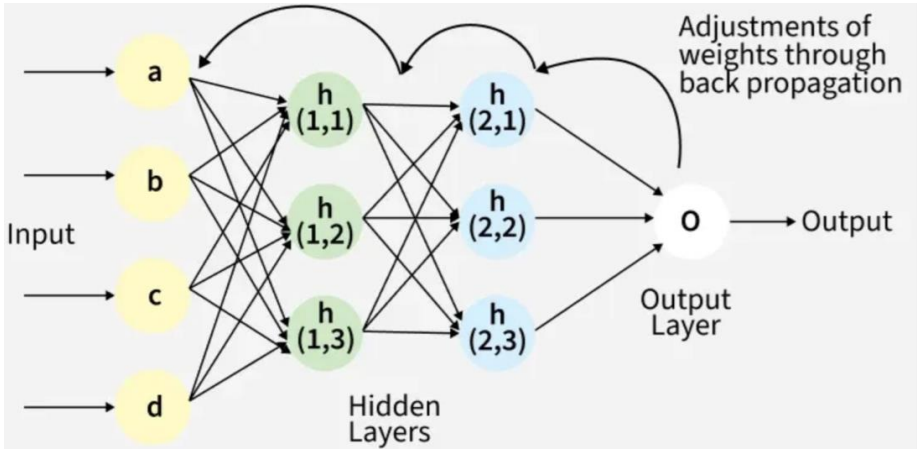
Source: Phillips, 2021

Learning (Training) a Neural Network

- How to *tune* the parameters so that the output matches the groundtruth (observations) as *close* as possible?
- Need to define a *loss function* to measure how *close*, e.g. MSE, cross-entropy loss
- **Back-propagation**: follow the negative gradient direction to find a minimizer!



$$f(\mathbf{w}+\mathbf{v}) \approx f(\mathbf{w}) + \nabla f(\mathbf{w})^\top \mathbf{v}.$$



Compute the gradient across the edge weights by chaining partial derivatives backwards through the network.

Example of a Neural Network

Consider a fully-connected neural network with structure $3 - 2 - 1$, where the input layer consists of 3 neurons, the hidden layer consists of 2 neurons, and the output layer consists of 1 neuron, with the network output

$$y = \mathrm{ReLU}(\mathbf{W}^{(2)} \cdot \mathrm{ReLU}(\mathbf{W}^{(1)} \cdot \mathbf{x}))$$

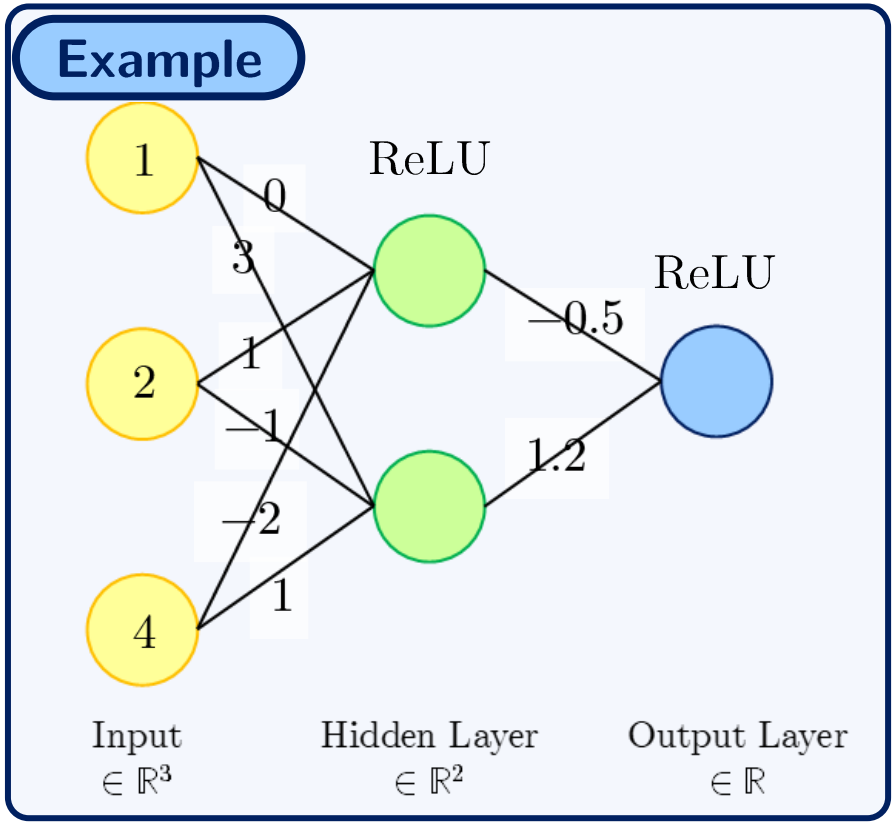
Weights:

$$\mathbf{W}^{(1)} = \begin{pmatrix} 0 & 1 & -2 \\ 3 & -1 & 1 \end{pmatrix}$$

$$\mathbf{W}^{(2)} = \begin{pmatrix} -0.5 & 1.2 \end{pmatrix}$$

Input:

$$\mathbf{x} = \begin{pmatrix} 1 \\ -2 \\ 4 \end{pmatrix}$$



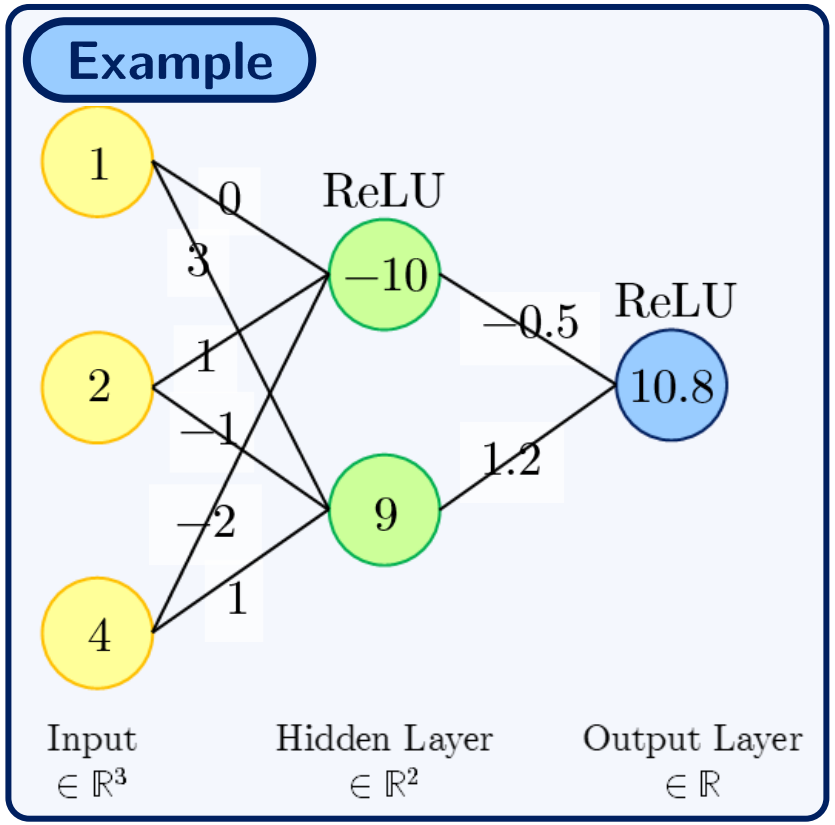
Example: Forward Propagation

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} = \begin{pmatrix} 0 & 1 & -2 \\ 3 & -1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ -2 \\ 4 \end{pmatrix} = \begin{pmatrix} 0(1) + 1(-2) + (-2)(4) \\ 3(1) + (-1)(-2) + 1(4) \end{pmatrix} = \begin{pmatrix} -10 \\ 9 \end{pmatrix}.$$

$$\mathbf{h}^{(1)} = \text{ReLU}\left(\mathbf{z}^{(1)}\right) = \begin{pmatrix} \max(0, -10) \\ \max(0, 9) \end{pmatrix} = \begin{pmatrix} 0 \\ 9 \end{pmatrix}.$$

$$z^{(2)} = \mathbf{W}^{(2)}\mathbf{h}^{(1)} = \begin{pmatrix} -0.5 & 1.2 \end{pmatrix} \begin{pmatrix} 0 \\ 9 \end{pmatrix} = (-0.5)(0) + (1.2)(9) = 10.8.$$

$$\hat{y} = h^{(2)} = \text{ReLU}\left(z^{(2)}\right) = \text{ReLU}(10.8) = \boxed{10.8}.$$



Example: Backward Propagation

- **Backpropagation** efficiently computes the gradient of the loss with respect to the network weights because it *reuses intermediate calculations from the output layer backward*, avoiding redundant operations.
- **Useful formulas**:

$$\boldsymbol{\delta}^{(\ell)} \triangleq \frac{\partial L}{\partial \mathbf{z}^{(\ell)}} \in \mathbb{R}^{n_\ell}$$

Error signal at layer ℓ

$$\boldsymbol{\delta}^{(L)} = \frac{\partial L}{\partial \mathbf{h}^{(L)}} \odot \left(\boldsymbol{\phi}^{(L)}\right)' \left(\mathbf{z}^{(L)}\right)$$

$$\boldsymbol{\delta}^{(\ell)} = \left(\mathbf{W}^{(\ell+1)}\right)^\top \boldsymbol{\delta}^{(\ell+1)} \odot \left(\boldsymbol{\phi}^{(\ell)}\right)' \left(\mathbf{z}^{(\ell)}\right) \quad \text{for } \ell = L-1, \ldots, 1$$

Hadamard product:

$$(\mathbf{A} \odot \mathbf{B})_{ij} = (\mathbf{A})_{ij}(\mathbf{B})_{ij}$$

$$\boxed{\frac{\partial L}{\partial \mathbf{W}^{(\ell)}} = \boldsymbol{\delta}^{(\ell)} \left(\mathbf{h}^{(\ell-1)}\right)^\top} \qquad \boxed{\frac{\partial L}{\partial \mathbf{b}^{(\ell)}} = \boldsymbol{\delta}^{(\ell)}}$$

Example: Backward Propagation

$$L = \frac{1}{2}(h^{(2)} - y_{\text{true}})^2, \qquad y_{\text{true}} = 9.$$

$$\frac{\partial L}{\partial h^{(2)}} = h^{(2)} - y_{\text{true}} = 10.8 - 9 = 1.8,$$

$$\left(\phi^{(2)}\right)'(z^{(2)}) = \text{ReLU}'(10.8) = 1.$$

$$\boxed{\boldsymbol{\delta}^{(2)} = 1.8 \times 1 = 1.8}.$$

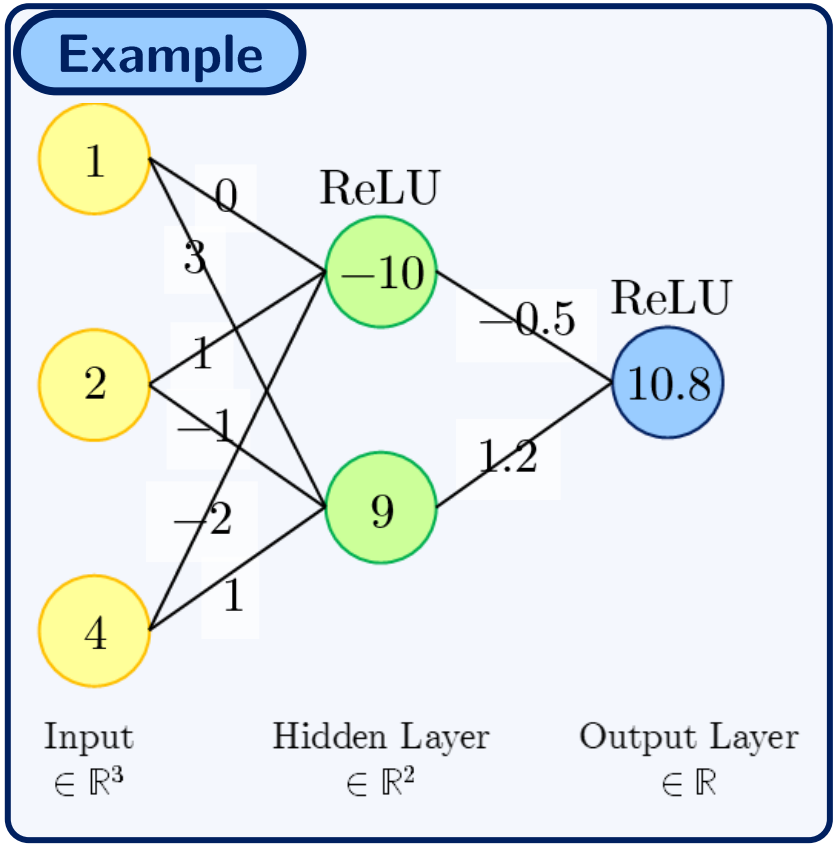
$$\left(\mathbf{W}^{(2)}\right)^\top \boldsymbol{\delta}^{(2)} = \begin{pmatrix} -0.5 \\ 1.2 \end{pmatrix} \times 1.8 = \begin{pmatrix} -0.9 \\ 2.16 \end{pmatrix},$$

$$\left(\phi^{(1)}\right)'\left(\mathbf{z}^{(1)}\right) = \text{ReLU}'\begin{pmatrix} -10 \\ 9 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

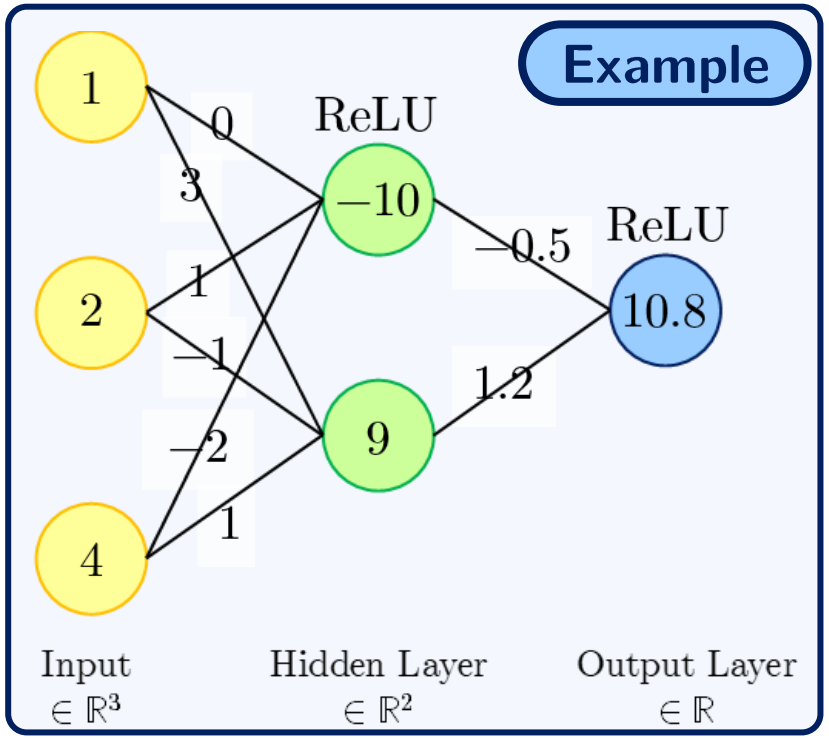
$$\boxed{\boldsymbol{\delta}^{(1)} = \begin{pmatrix} -0.9 \\ 2.16 \end{pmatrix} \odot \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 2.16 \end{pmatrix}}.$$

$$\boxed{\frac{\partial L}{\partial \mathbf{W}^{(2)}} = \boldsymbol{\delta}^{(2)}\left(\mathbf{h}^{(1)}\right)^\top = 1.8 \times \begin{pmatrix} 0 & 9 \end{pmatrix} = \begin{pmatrix} 0 & 16.2 \end{pmatrix}}.$$

$$\boxed{\frac{\partial L}{\partial \mathbf{W}^{(1)}} = \boldsymbol{\delta}^{(1)}\left(\mathbf{h}^{(0)}\right)^\top = \begin{pmatrix} 0 \\ 2.16 \end{pmatrix} \times \begin{pmatrix} 1 & -2 & 4 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 \\ 2.16 & -4.32 & 8.64 \end{pmatrix}}$$



Example: Gradient Descent Update



$$\boxed{\frac{\partial L}{\partial \mathbf{W}^{(2)}} = \boldsymbol{\delta}^{(2)} \left(\mathbf{h}^{(1)}\right)^\top = 1.8 \times \begin{pmatrix} 0 & 9 \end{pmatrix} = \begin{pmatrix} 0 & 16.2 \end{pmatrix}}.$$

$$\boxed{\frac{\partial L}{\partial \mathbf{W}^{(1)}} = \boldsymbol{\delta}^{(1)} \left(\mathbf{h}^{(0)}\right)^\top = \begin{pmatrix} 0 \\ 2.16 \end{pmatrix} \times \begin{pmatrix} 1 & -2 & 4 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 \\ 2.16 & -4.32 & 8.64 \end{pmatrix}}$$

Take a gradient step with learning rate $\eta = 0.1$

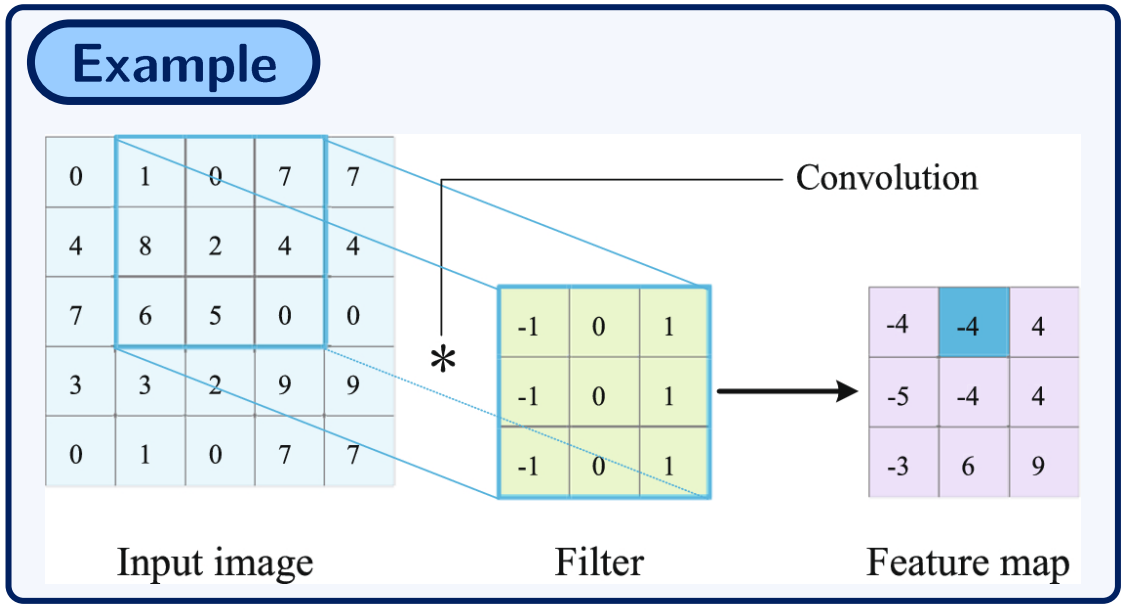
$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial L}{\partial \mathbf{W}^{(l)}}, l = 1 \ldots \ell$$

$$\mathbf{W}^{(2)}_{\text{new}} = \mathbf{W}^{(2)} - \eta \frac{\partial L}{\partial \mathbf{W}^{(2)}} = \begin{pmatrix} -0.5 & 1.2 \end{pmatrix} - 0.1 \begin{pmatrix} 0 & 16.2 \end{pmatrix} = \boxed{\begin{pmatrix} -0.5 & -0.42 \end{pmatrix}}.$$

$$\mathbf{W}^{(1)}_{\text{new}} = \mathbf{W}^{(1)} - \eta \frac{\partial L}{\partial \mathbf{W}^{(1)}} = \begin{pmatrix} 0 & 1 & -2 \\ 3 & -1 & 1 \end{pmatrix} - 0.1 \begin{pmatrix} 0 & 0 & 0 \\ 2.16 & -4.32 & 8.64 \end{pmatrix} = \boxed{\begin{pmatrix} 0 & 1 & -2 \\ 2.784 & -0.568 & 0.136 \end{pmatrix}}$$

Convolutional Neural Networks (CNN)

- Use case: when data has a **grid-like structure** where the arrangement, ordering, and local proximity of the elements matter.
- Example: Pollution and meteorology are spatially structured – neighboring grid cells are correlated. Time axis are also correlated, leading to forecasting applications.
- Uses sparse interactions where a small filter looks at only a local patch of the input at a time. This drastically reduces memory usage and total parameter count.

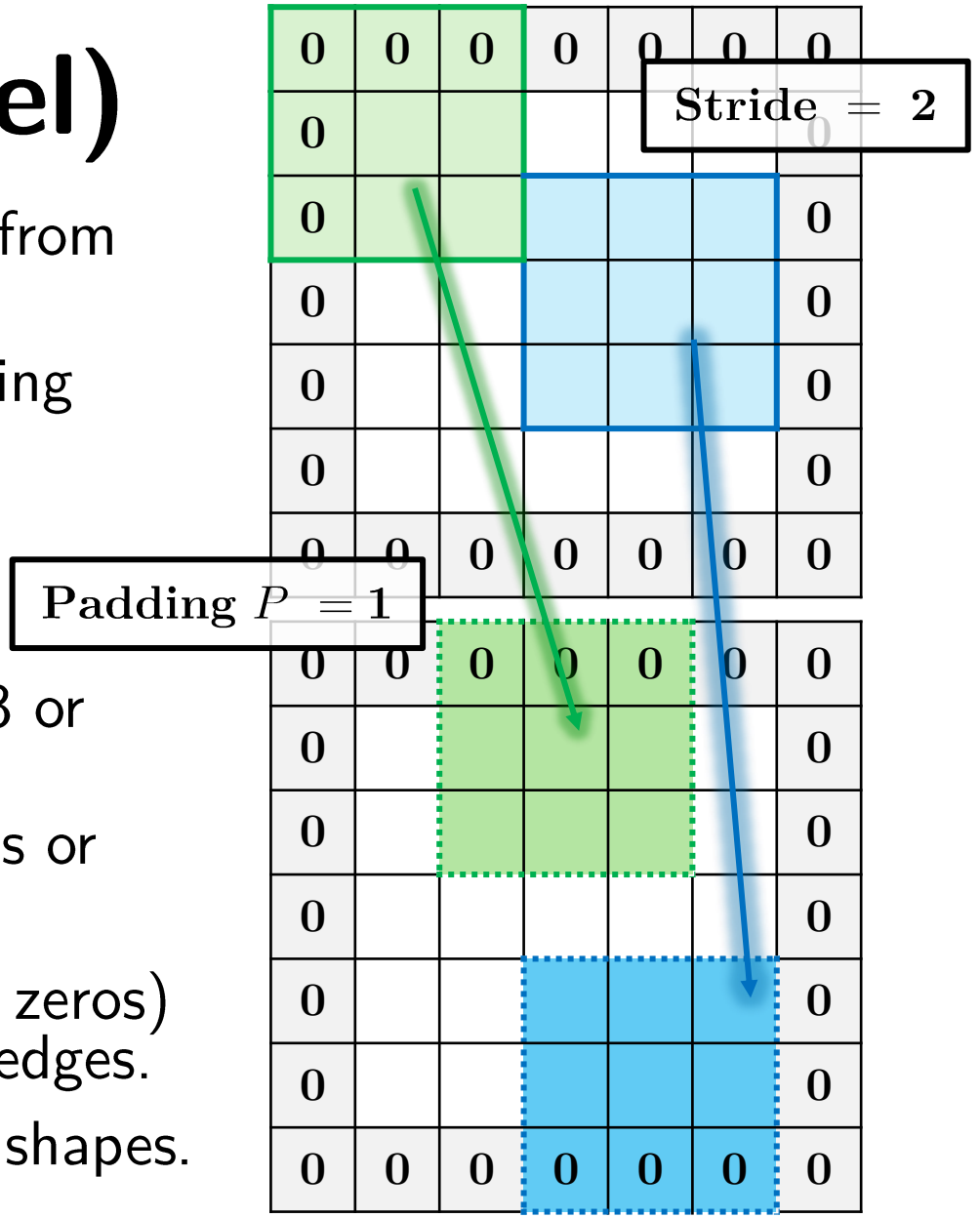


Convolutional Filter (Kernel)

- The kernel K slides across the input image pixels from left to right, top to bottom.
- It multiplies its own weight values by the overlapping image pixel values.

$$s_{i,j} = \phi\left(\sum_{m=0}^{M-1}\sum_{n=0}^{N-1} X'_{i\cdot S+m, j\cdot S+n} \cdot K_{m,n} + b\right)$$

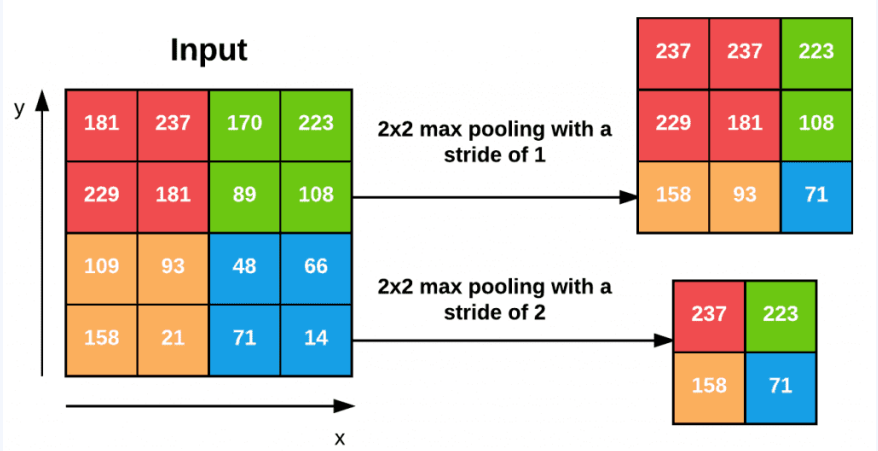
- **Size ($M \times N$)**: Usually small odd numbers like 3x3 or 5x5 to have a clear center pixel.
- **Stride (S)**: The number of pixels the kernel moves or hops at a time.
- **Padding (P)**: Adding extra border pixels (usually zeros) around the image so the kernel can scan the very edges.
- Learn and spot patterns, e.g. edges, textures, and shapes.



Pooling Layer for Down-sampling

- Pooling layer down-samples the volume spatially, independently in each depth slice of the input volume. Hence, the volume depth (i.e., feature channels) is preserved.
- Keeping only dominant features and helps later layers in the networks see a broader context of the overall image. Ensures scale-invariance, and minor shift/rotational-invariance.
- In this example, the input volume of size $[4\times4\times1]$ is pooled with filter size 2, stride 2 into output volume of size $[2\times2\times1]$. Each max is taken over 4 numbers (the little 2×2 squares).

Example



$$p_{ij} = \max_{(m,n)\in R_{ij}} a_{mn}$$

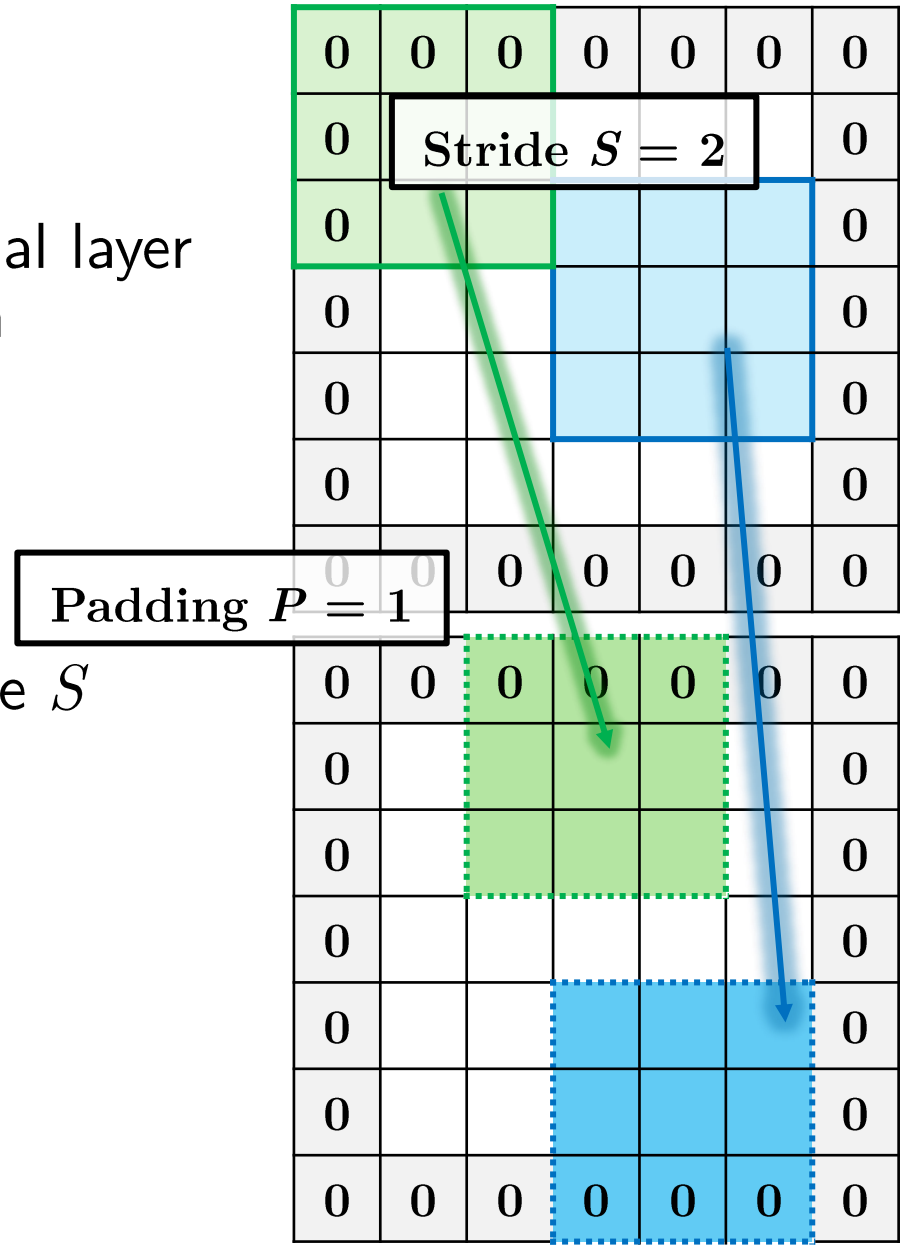
Size of the Output

For a 3D input with size $W_1 \times H_1 \times D_1$, the convolutional layer with K filters, spatial extent F and stride S produces a volume with size $W_2 \times H_2 \times D_2$, where

$W_2 = \lfloor (W_1 + 2P - F)/S \rfloor + 1$,
$H_2 = \lfloor (H_1 + 2P - F)/S \rfloor + 1$,
and $D_2 = K$.

Instead, a pooling layer with spatial extent F and stride S produces a volume with size $W_2 \times H_2 \times D_2$, where

$W_2 = \lfloor (W_1 - F)/S \rfloor + 1$,
$H_2 = \lfloor (H_1 - F)/S \rfloor + 1$,
and $D_2 = K$



CNN in Practice

Input field
Gridded maps: meteorology, emissions

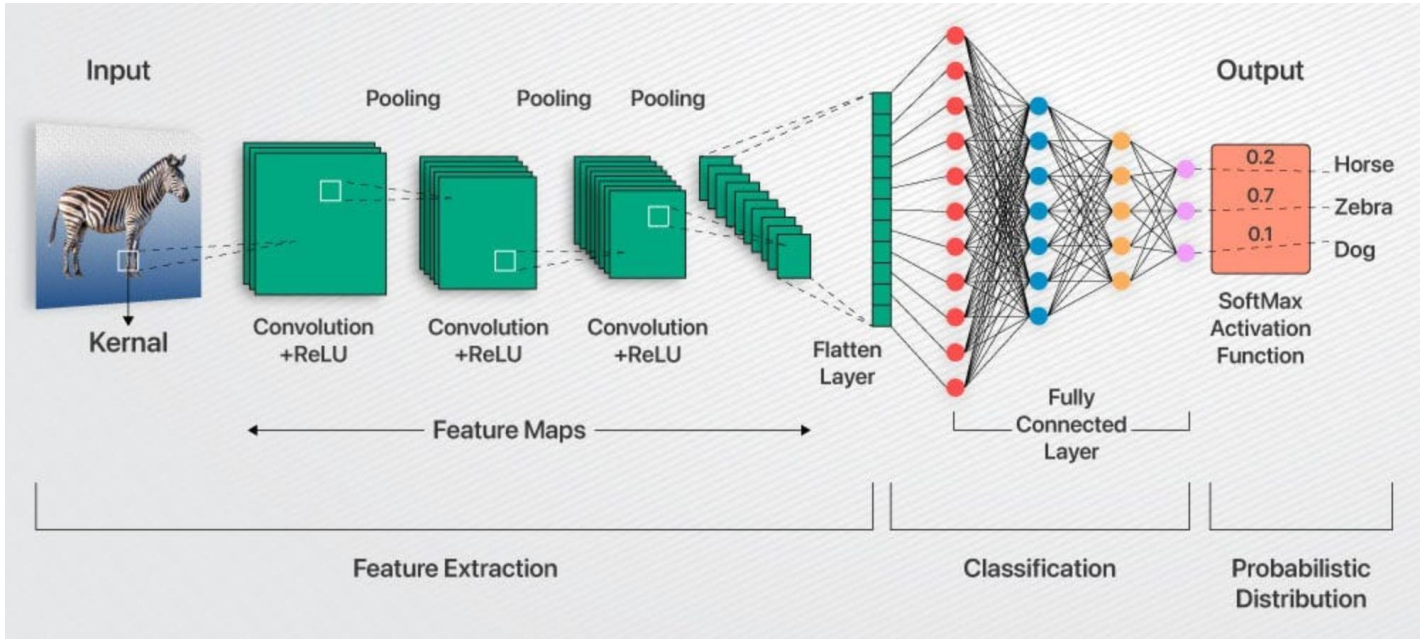
→ **Conv + ReLU**
Detect local spatial patterns

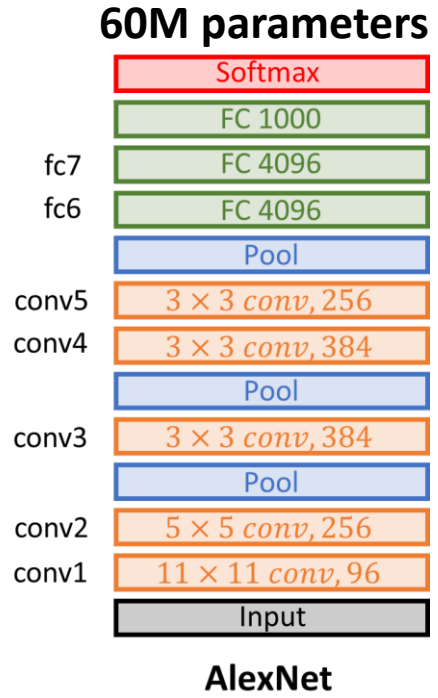
→ **Pooling**
Downsample, keep strong signals

→ **Conv + Pool**
Deeper, more abstract features

→ **Dense**
Combine into prediction

→ **O_3 forecast**
Continuous output





SHapley Additive exPlanations (Lundberg and Lee, NIPS 2017)

Game-theoretic approach: how to fairly divide the total reward among a group based on each member's contribution?

Given a set of players $N=[n]$, a coalition $S \subseteq N$ is any subset of players. The Shapley value for a player i in a game defined by a characteristic function $v:2^N \to \mathbb{R}$ is given by

$$\varphi_i(v) = \frac{1}{n!} \sum_{S \subseteq N \setminus \{i\}} |S|! \cdot (n - |S| - 1)! \cdot (v(S \cup \{i\}) - v(S))$$

average | multiplicity | coalition value | contribution value

Idea: Average the *marginal contributions* over **all possible orders**

How to attribute a prediction of the model to a single feature?

Let $v(S)$ be the model's expected prediction when only features in S are "*known*", i.e.

$$v(S) = \mathbb{E}_{X_{\bar{S}}}[f(\mathbf{x}_S, X_{\bar{S}})]$$

Applications

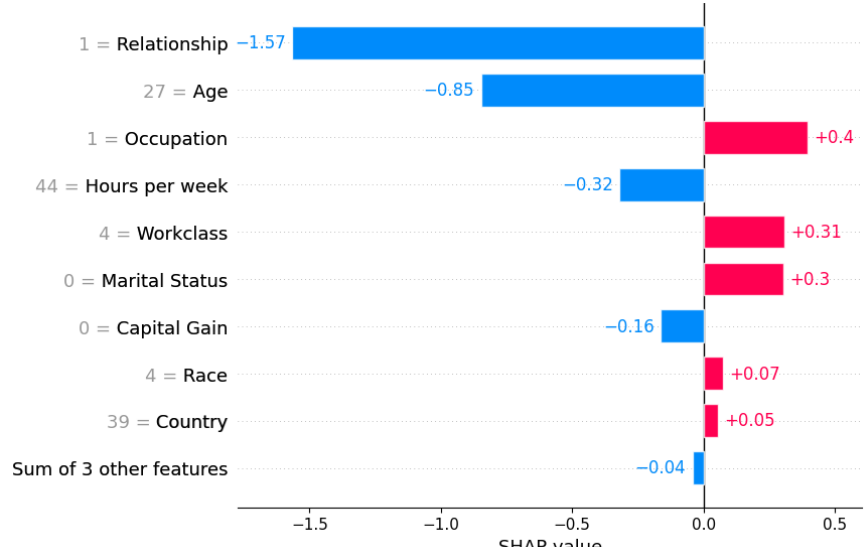
- Explaining an ML **emulator** and identifying drivers of a climate or weather model component.
- Understanding **extreme event** predictions such as wildfires, heatwaves, or floods.
- Detecting pollution sources or transport-weather interactions.
- Debugging models and building trust with domain scientists.

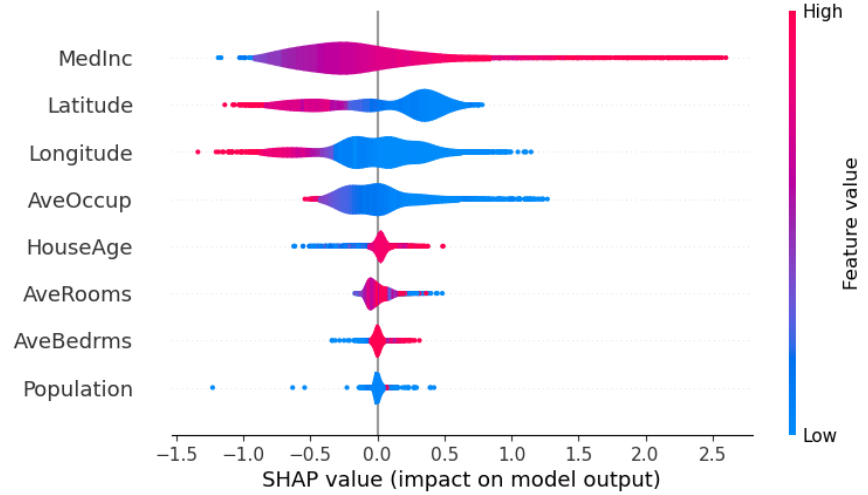
SHapley Additive exPlanations (Lundberg and Lee, NIPS 2017)

- SHAP fairly distributes the prediction among the features by averaging a feature's marginal contribution over all possible subsets of other features.

$$f(\mathbf{x}) = \phi_0 + \sum_i \phi_i$$

  - ϕ_0 = baseline prediction, the average model output.
  - ϕ_i = SHAP value for feature i.
  - Positive ϕ_i pushes the prediction higher.
  - Negative ϕ_i pushes it lower.
- In practice, implementations such as `TreeSHAP` approximate this efficiently.





Building Trustworthy AI/ML Tools

- Trustworthy AI, explainable AI and physics-informed machine learning.
- Ways to incorporate physical knowledge into machine learning models:
  - Change the architecture of the neural network
  - Formulate the machine learning problem in a physical way
  - Close the budget for the output variables
  - Correct the outputs to fulfil the constraint
  - Incorporate physical constraints into the loss function
- Evaluate the machine learning solution for reproducing the correct physics
  - Consider specific use cases and weather regimes
  - Perform sensitivity tests on the inputs or outputs
  - Test for physical reasoning (e.g. for extreme events)
  - Benchmarking on existing solutions

A High-Level View of How ML/AI Will Be Used in Earth System Science

- **Improve understanding:** fuse data from multiple sources, use unsupervised learning and causal discovery, enable AI-powered visualization, and quantify uncertainty.
- **Speed up simulations and green computing:** emulate model components, port emulators to heterogeneous hardware, use reduced precision and sparse ML, optimize HPC/data workflows, and compress data.
- **Improve models:** learn model components from observations, correct biases, perform quality control on observations/operators, and detect features.
- **Link communities:** apply Earth system science to health risks, energy downscaling, transport with weather/IoT data, pollution source detection, and extreme events like wildfires.

References

Choi, P. T. (2025). *MATH3320: Foundation of Data Analytics* [Lecture slides]. Department of Mathematics, The Chinese University of Hong Kong. https://www.math.cuhk.edu.hk/course/2526/math3320

Christopoulos, C. D., Garimella, S., Zawadowicz, M. A., Möhler, O., & Cziczo, D. J. (2018). A machine learning approach to aerosol classification for single-particle mass spectrometry. *Atmospheric Measurement Techniques*, *11*(10), 5687-5699.

Ho, K. C., Minh, V. T. T., Tin, T. T., & Hien, T. T. (2021). PM2.5 forecast system by using machine learning and WRF model, A case study: Ho Chi Minh City, Vietnam. *Aerosol and Air Quality Research*, 21(12), 210108.

Ho, T. K. (1995). "Random decision forests." *Proceedings of 3rd International Conference on Document Analysis and Recognition*, 1, 278-282. doi: 10.1109/ICDAR.1995.598994

Koss, A. R., Canagaratna, M. R., Zaytsev, A., Krechmer, J. E., Breitenlechner, M., Nihill, K. J., Lim, C. Y., Rowe, J. C., Roscioli, J. R., Keutsch, F. N., and Kroll, J. H. (2020). Dimensionality-reduction techniques for complex mass spectrometric datasets: application to laboratory atmospheric organic oxidation experiments, *Atmospheric Science and Physics*, 20(2), 1021–1041.

Lee, C. J., Makar, P. A., & Soares, J. (2026). Clustering Analysis of Very Large Measurement and Model Data Sets on High-Performance Computing Platforms. *Journal of Geophysical Research: Atmospheres*, 131(1), e2025JD043822.

Lundberg, S. M., & Lee S.-I. (2017). A unified approach to interpreting model predictions. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17)*. 4768–4777.

McGovern, et al. (2019). Making the Black Box More Transparent: Understanding the Physical Implications of Machine Learning. *Bulletin of the American Meteorological Society, 100*(11), 2175-2199.

Nath, P., Moss, H., Shuckburgh, E., & Webb, M. (2024). RAIN: Reinforcement Algorithms for Improving Numerical Weather and Climate Models. In *NeurIPS 2024 Workshop on Tackling Climate Change with Machine Learning*. **Climate Change AI**. *arXiv:2408.16118v3*.

Phillips, J. M. (2021). *Mathematical foundations for data analysis*. Springer.

Rajesh, M., Babu, R. G., Moorthy, U., & Easwaramoorthy, S. V. (2025). Machine learning-driven framework for realtime air quality assessment and predictive environmental health risk mapping. *Scientific Reports*.

Reichstein, M. et al. Deep learning and process understanding for data-driven Earth system science. *Nature 566*, 195–204 (2019).

Sinha, J. D., Gupta, S., Samanta, S., Dash, B. B., De, U. C., & Patra, S. S. (2025). Intelligent Air Quality Control Through Continuous Policy Learning. *2025 9th International Conference on Inventive Systems and Control (ICISC)*.

Zaidan, M. A. et al. (2025). "Artificial intelligence for atmospheric sciences: A research roadmap." *arXiv preprint. arXiv:2506.16281*.